

Embedded Computer Vision Final Report

Phil Orlando
Graduate Student, Department of Electrical Engineering
University of Colorado Boulder

All source code used for this assignment is available at <https://github.com/Pmorlando/ECV-Orlando/tree/main/FinalProj> and if issues downloading or viewing are encountered, please contact me. I am not the most experienced with Github but happy to learn of mistakes I am making.

For my final project, I chose to do a creative project which is a Black Jack assistant. The goal of the Black Jack assistant was to read a video I recorded with a minimum three cards where two are for the player and one is for the dealer. The Black Jack assistant will read the video and in each frame, correctly identify the amount of cards seen using their outer contour, and identify which cards are present on the screen and display the values overlaid with the card outline on the current frame with a recommendation for the play to either hit, stand, or double down based on the dealer's show card and the player's hand. For my project proposal, in the minimum requirements, the feature is multiple card recognition, 10 frames per second (FPS) of throughput and precision (P) of having correct guesses for card values to be over 0.8 and recall (R) where identification cards present in the video are found is greater than 0.8. The target features are that the cards are properly identified as player and dealer cards along with their associated Black Jack value being found and displayed on the display with the same card identification as the minimum. Additionally, the P and R values increase to greater than 0.9 and throughput performance is increased to 20 FPS. Finally, optimal performance feature is the recommendation for the player to hit, stand or double down displayed and the ability to handle more cards being dealt to the player beyond the first two. The throughput performance increases to 30 FPS and P and R are greater than 0.95.

To begin designing and testing this project, I started identifying the key features to make the Black Jack assistant work. I started working with still images at 1080p quality and working through finding cards on the screen. To do this, I relied on previous methods from this course for processing the image including blur, canny edge detection and then to find the cards from the canny edge detection image using contours. To find an appropriate value for the Canny edge detection lower threshold, I used the Canny edge detection file with the slider to find a value appropriate to the card setup I was using.

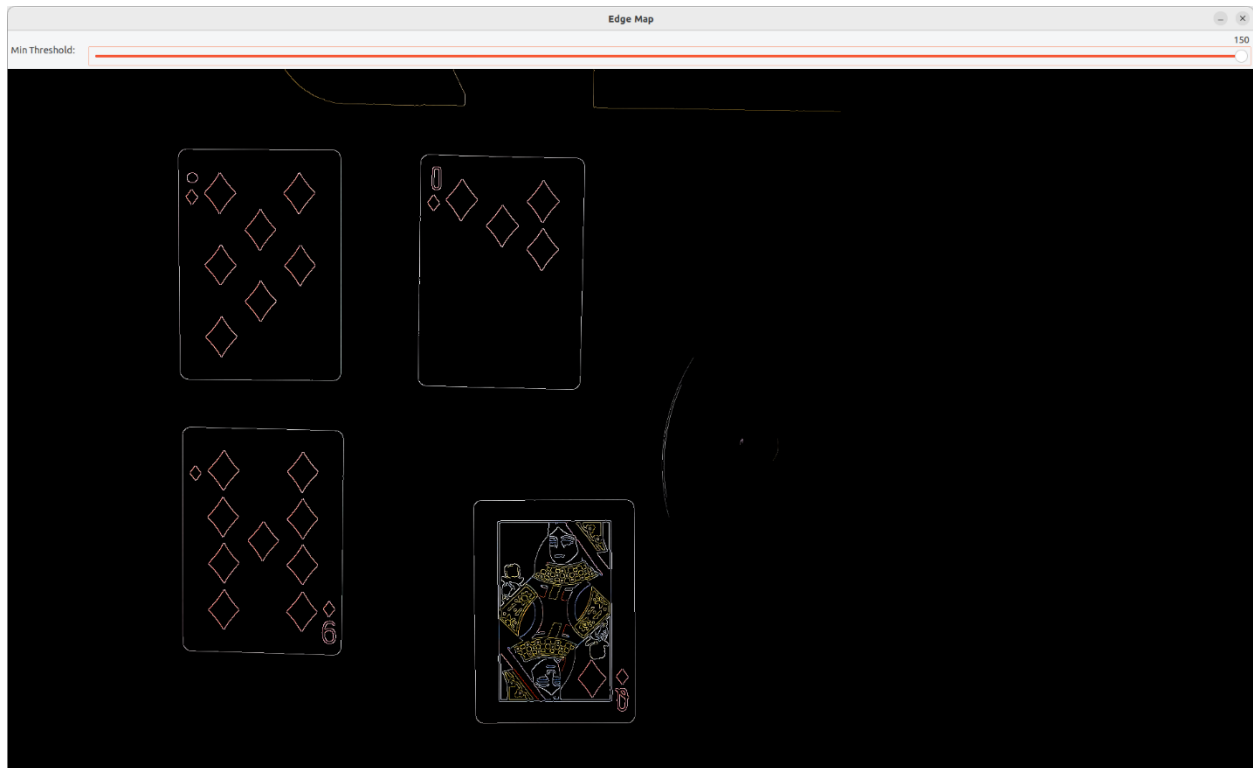


Figure (1) Canny edge detection threshold testing.

I had seen this method in examples online I researched so I know I was at least on the right path [1]. Once I was able to find the contours on the entire image, I noticed with four cards on the still image there were four contours sized right around the same 80000 pixels. By setting a minimum and maximum threshold value I was able to determine how many cards were in the image based on how many contours are in the specified range [2]. To reduce accidental identification of other objects, I recorded the test videos and images myself in a clear portion of my best beneath a lamp to have consistent lighting. Where I began to divert from other card recognition methods I found researching was the fact that the Black Jack value of a card does not consider the suit of the cards. Therefore, I would not need to detect cards and rather just the number value in the corner of the card.



Figure (2) Card template example

I found OpenCV match template function to be exactly what I needed for this [3]. The match template function takes a template and by sliding it across the image it is compared against, similar to a point spread function, if there is significant overlap between the template and image pixels, a spike in the correlation will be found. Because color images are not significant to this comparison, I took the grayscale of the original card image for all processing. To test the concept

and current code structure, I started with four card templates of the top left corners of cards and scanning each template against the entire image of those four cards on my desk. The match template only outputs the highest correlation and where that is found on the image. To troubleshoot, I have print statements to the correlation value found for each card and a rectangle drawn over where each highest correlation is. Using this with a singular template image and slightly skewed cards resulted in imperfect matches and a nine being found on the bottom right corner of a six.

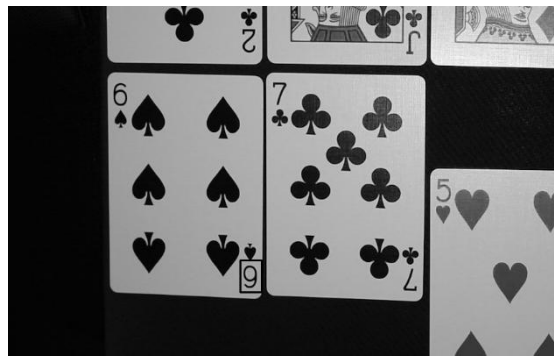


Figure (3) Upside down 6 recognized as a 9.

To ensure that multiple cards of the same value can get recognized in an image, I used the contours to isolate individual cards found in the image to be used for the comparison against the templates. This is necessary for the final product but adds more computation per image by having to run each card against all templates. To reduce computations so I can meet the performance metrics and knowing that suit is not necessary for Black Jack, I only created 13 templates with one for each card instead of 52 templates reducing 75% of the total comparisons for every card.

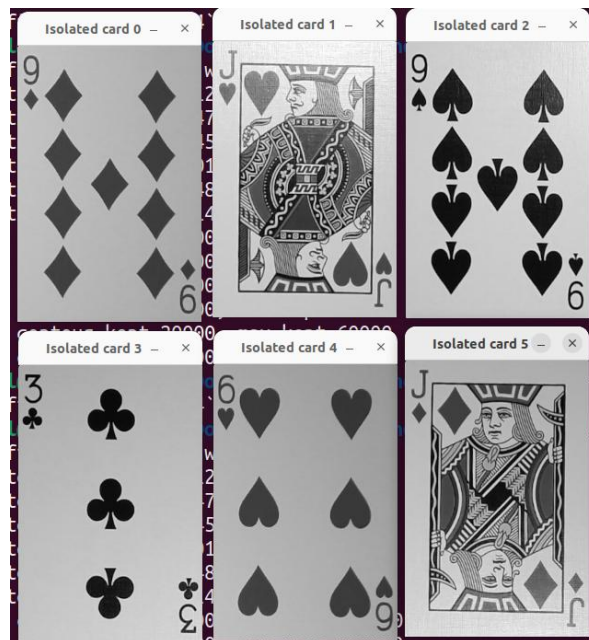


Figure (4) Isolated cards after warping straight.

After more testing where each isolated card was compared against all 13 templates to find the highest correlation, I was finding some lower correlation values than I was expecting for seemingly cards with the same lighting I realized that the cards may be slightly canted and because the templates are not rotated during the comparison there may not be as high if a correlation as expected. Therefore, I would need to straighten the isolated cards even if they are not perfectly straight up and down. To do this, I needed the four corners of the card but the contours instead of an output of continuous points, it uses two endpoints for straight lines, but the corners of the contour are many close points. I used `approxPolyDP()` to take the contour points and connect the closest points at the rounded corners to end up with four corner points [4].

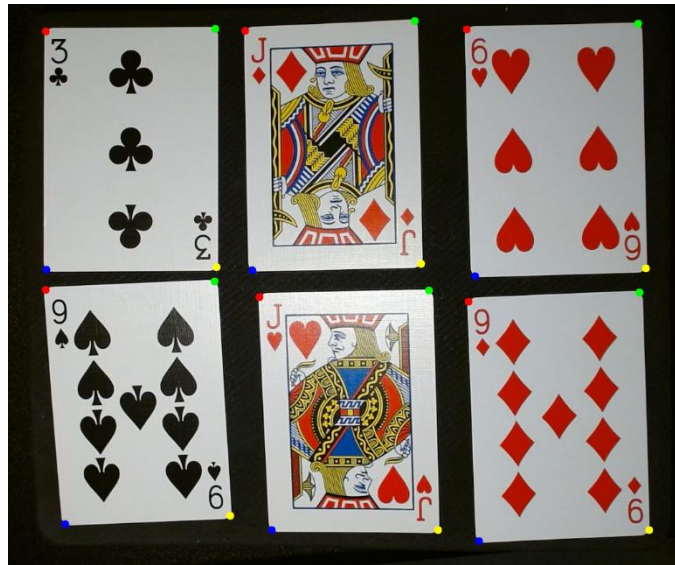


Figure (5) Card corner points after `approxPolyDP()`.

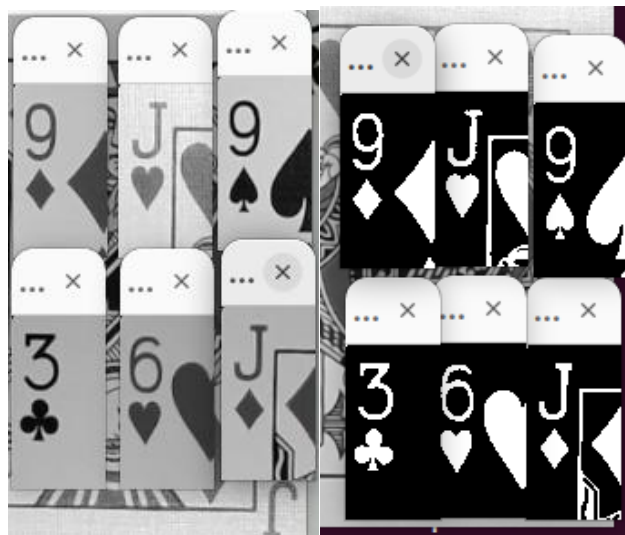


Figure (6) Top left corner of cards for matching before and after binary threshold.

Then to correct any angle of the cards, using the four corner points and then four points of a 300x200 rectangle and finding the linear transformation from the card's corners to the rectangle points. Then using the linear transformation, I warped the cards corners to the rectangle straightening it out [5]. Using the straightened card, I cropped the top left corner to remove the six and nine being seen on the bottom right corner. And having less space per card to need the match template to compute for. The templates I created for every card are only the top left corner showing the card value. This, however, still had issues where cards were being matched to a different card. I noticed that the grayscale images for a black card and red card were not the same pixel values on the number with the black cards being higher pixel values. This affects the matching where the difference in pixel value even over the same area will not output as high of a correlation. To account for this, I used thresholding to set the card template and card image pixel values to either 255 or 0. Using OpenCV's threshold function with the flag binary so that it would set the values to 0 or 255 and then also the Otsu flag. Otsu is an automatic threshold algorithm that finds a cutoff point of an image for thresholding based on analyzing the histogram of the image [6]. After using this for the comparison on the still frames, I was getting much better results but still a few cards were not being recognized, or others were constantly creating false positives. To correct this, I used every card in the deck to create 4 templates for each card value. This would increase the comparisons per frame but was necessary for getting accurate card predictions. Using images of different lighting and even if the thresholding was not perfect, there were two templates per card from red and black cards. Once I was able to get thresholding and the multiple templates, I was able to correctly identify every card in all of my test images.

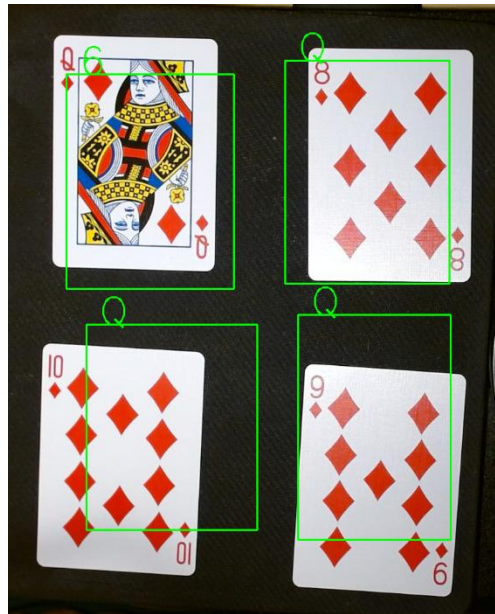


Figure (7) Bad matching for cards.

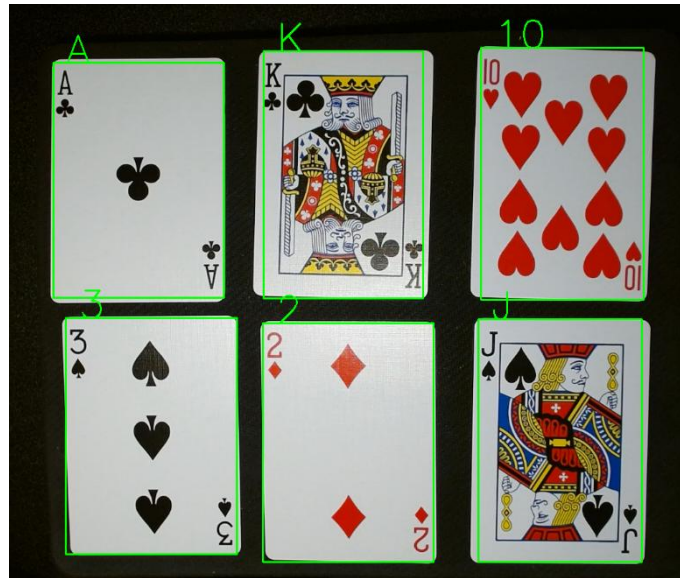


Figure (8) Correct matching for all 6 cards on screen.

Before moving to video, I wanted to test the speed each frame would take to process, take the cards found and compare against the templates. Using syslog and monotonic clock timestamps, I was calculating the total process time for six cards in an image to be around 150 ms with 90 ms for the matching and 30 ms for the processing. This would result in around 14 FPS which was below my requirements although I know this was entirely correctable. The immediate way I knew I could get a faster run time was to use threading for the matching. Each card found in an images is matching to all 13 cards and all four templates for each card. Because the next step in the process is to display the results, this could very much benefit from threading. Although I was testing with six cards from the image, I know the real tests are with three cards and then maybe a few more so assigning a core to each card found would be wasting the processing power of the six cores the Nvidia Jetson Nano has. To use OpenMP, I used a for loop with the amount of loops equal to the number of cards in the image multiplied by the 13 card values [7]. This way OpenMP can employ the threads most efficiently and use all cores for any number of cards. After threading the matching, with six cards I was able to cut the matching time from 90 ms to 20 ms.

```

Jul 24 22:54:35 phil-desktop BlackJack_cv[286979]: frame process time 154.612 ms, WCET FPS 6.468
Jul 24 22:54:35 phil-desktop BlackJack_cv[286979]: Template loading time 4.563 ms
Jul 24 22:54:35 phil-desktop BlackJack_cv[286979]: image loading time 26.512 ms
Jul 24 22:54:35 phil-desktop BlackJack_cv[286979]: contour process time 33.883 ms
Jul 24 22:54:35 phil-desktop BlackJack_cv[286979]: match process time 89.653 ms
Jul 24 22:54:47 phil-desktop BlackJack_cv[286988]: frame process time 152.046 ms, WCET FPS 6.577
Jul 24 22:54:47 phil-desktop BlackJack_cv[286988]: Template loading time 4.855 ms
Jul 24 22:54:47 phil-desktop BlackJack_cv[286988]: image loading time 27.693 ms
Jul 24 22:54:47 phil-desktop BlackJack_cv[286988]: contour process time 28.501 ms
Jul 24 22:54:47 phil-desktop BlackJack_cv[286988]: match process time 90.998 ms
Jul 24 22:54:49 phil-desktop BlackJack_cv[286995]: frame process time 160.133 ms, WCET FPS 6.245
Jul 24 22:54:49 phil-desktop BlackJack_cv[286995]: Template loading time 4.699 ms
Jul 24 22:54:49 phil-desktop BlackJack_cv[286995]: image loading time 29.209 ms
Jul 24 22:54:49 phil-desktop BlackJack_cv[286995]: contour process time 32.894 ms
Jul 24 22:54:49 phil-desktop BlackJack_cv[286995]: match process time 93.330 ms
Jul 24 22:54:52 phil-desktop BlackJack_cv[287002]: frame process time 152.312 ms, WCET FPS 6.565
Jul 24 22:54:52 phil-desktop BlackJack_cv[287002]: Template loading time 4.634 ms
Jul 24 22:54:52 phil-desktop BlackJack_cv[287002]: image loading time 27.404 ms
Jul 24 22:54:52 phil-desktop BlackJack_cv[287002]: contour process time 28.914 ms
Jul 24 22:54:52 phil-desktop BlackJack_cv[287002]: match process time 91.359 ms
Jul 24 22:54:53 phil-desktop BlackJack_cv[287009]: frame process time 150.884 ms, WCET FPS 6.628
Jul 24 22:54:53 phil-desktop BlackJack_cv[287009]: Template loading time 4.564 ms
Jul 24 22:54:53 phil-desktop BlackJack_cv[287009]: image loading time 27.521 ms
Jul 24 22:54:53 phil-desktop BlackJack_cv[287009]: contour process time 28.135 ms
Jul 24 22:54:53 phil-desktop BlackJack_cv[287009]: match process time 90.663 ms
phil@phil-desktop:~/Documents/ECV-Drlande/FinalProj$
  
```

Figure (9) Timing breakdown before optimization.

Following that, in my early testing the input images were 1080p because the more defined features of a higher quality image would help the matching. However, the 30 ms image processing time was much too long. I therefore compressed the input image for the blur, Canny edge detection, and contour finding and then used the contours found and reimposed that to the original image and isolated the individual cards that way. This improved the processing time from 30 ms to 17 ms. This was only around 15 FPS, but I was using non optimized compile commands along with processing and image with six cards instead of the expected three so this is a fairly reasonable WCET that should average at a much faster run time.

```
Jul 24 22:52:16 phil-desktop BlackJack_cv[286908]: frame process time 70.130 ms, WCET FPS 14.259
Jul 24 22:52:16 phil-desktop BlackJack_cv[286908]: Template loading time 4.572 ms
Jul 24 22:52:16 phil-desktop BlackJack_cv[286908]: image loading time 27.583 ms
Jul 24 22:52:16 phil-desktop BlackJack_cv[286908]: contour process time 17.644 ms
Jul 24 22:52:16 phil-desktop BlackJack_cv[286908]: match process time 20.330 ms
phil@phil-desktop:~/Documents/ECV-Orlando/FinalProj$
```

Figure (9) Timing breakdown after threading and input compression.

I then started testing with 30 FPS video and then use timestamps to calculate the actual FPS across the entire video. I compiled with the -O0 flag which is not optimized. I filmed six sets of test videos, three in 1080p and three in 720p to test both image qualities. I first started testing with 720p still images and no compression for processing input images and only incorrectly identified two cards across 27 cards. Therefore, I decided to use the 720p videos and see the speeds I could reach without compiling with the -O3 flag. If the first few rounds of video testing, the cards were properly predicted for the majority of the frames as I watched live. Then I saw that the overall throughput speed was just under 30 FPS. I then tested for all three 720p videos and was testing right around 30 FPS for each video. After confirming the speed is near the optimal range, I added a wait key to go frame by frame to analysis the precision and recall. To test the reliabvility data, I went frame by frame for a single round through each of the three videos. To give the benefit of the doubt where my had is covering some portion of the card, I did not count those frames as false negatives. Across all videos I had approximately 10 false negatives. In the first video I had approximately 39 false positives. In video 2, I had 6 false positives and in video 3 I had approximately 40 false positives. Overall I averaged above 0.95 for precision and 0.99 for recall.

References:

- [1] <https://www.youtube.com/watch?v=m-QPjO-2IkA>
- [2] <https://learnopencv.com/contour-detection-using-opencv-python-c/#Steps-for-Finding-and-Drawing-Contours-in-OpenCV>

- [3] <https://www.funvisiontutorials.com/2023/07/cvmatchtemplate-function-in-opencv-to.html>

- [4] https://docs.opencv.org/4.13.0/d3/dc0/group__imgproc__shape.html#ga0012a5fdae70b8a9970165d98722b4c

- [5] <https://www.geeksforgeeks.org/python/perspective-transformation-python-opencv/>

- [6] <https://opencv.org/image-thresholding-using-opencv/>

- [7] <https://stackoverflow.com/questions/11773115/parallel-for-loop-in-openmp>

- [8] <https://www.youtube.com/watch?v=INFIurqyNUM>

- [9] https://www.researchgate.net/publication/366868685_Implementing_Playing_Cards_BlackJack_Game_using_OpenCV